

CCL Generation and Caseworker Template Management Guide

Comprehensive Operational & Technical Reference Guide for Client Care Letter (CCL) Workflows, Template Engineering, Security & RBAC Policies

Application:	ImCamHub / ElitePic CRM & UAT Portal
Document Version:	2.1 (Production Standard)
Date:	September 2026
Classification:	Internal Technical & Business Operational Standard
Target Audience:	Caseworkers, Supervisors, Admins, QA, Product, Engineering
Status:	FIXED & VERIFIED (Code & Test Audited)

Table of Contents

1. Document Overview
2. Complete CCL Generation Workflow
3. CCL Data Sources & Tag Mapping
4. CCL Template Architecture
5. How the CCL Template is Rendered
6. Caseworker CCL Template Management User Guide
7. Role and Permission Matrix (RBAC)
8. Security and Data Isolation Policies
9. API Endpoint Reference
10. Database Schema and File Storage Model
11. Error Handling and Troubleshooting Matrix
12. QA Testing Checklist & Test Results
13. UI Screenshots & Empirical Evidence
14. Implementation Gaps & Strategic Recommendations
15. Final Executive Summary

1. Document Overview

This document provides the authoritative technical, architectural, and operational reference for the Client Care Letter (CCL) generation and template management subsystem within the ImCamHub / ElitePic CRM application. Every workflow, data mapping, RBAC check, API route, and database interaction described herein has been directly audited from the application source code and verified against live automated regression suites.

1.1 What CCL Means in ImCamHub CRM

In ImCamHub CRM, CCL stands strictly for Client Care Letter. It is a legally binding client engagement document issued by the legal firm / organisation to a Candidate client prior to commencing work on an immigration case. The letter outlines the agreed legal fee, payment installment schedule, scope of immigration advice, caseworker assignment, and client terms of service.

1.2 Why CCL is Generated & Case Lifecycle Gate

The Client Care Letter serves as the mandatory Submission Gate in the immigration case lifecycle. Under the system's workflow engine (`assertSubmissionGate`), an immigration case cannot transition to the `'application_submitted'` stage until the CCL has been officially issued by

a Caseworker or Admin, accepted/signed by the Candidate client, and the required initial fee payment is settled.

1.3 Who Interacts with CCL Workflows

- Caseworkers & Admins: Create, edit, preview, import (.docx), and issue Client Care Letters for assigned cases. Manage visa-type specific templates.
- Candidate Clients: Receive in-app and email notifications when a CCL is issued, view the letter, accept/sign the letter via the Client Portal, and download the PDF.
- Superadmins & Supervisors: Manage firm-wide templates, audit custom fee proposals, and oversee submission gates.



CORE SYSTEM SPECIFICATION

CCL is NOT a generic document upload. It is an automated, dynamic document generated via HTML/CSS templates and compiled into PDF format via pdfmake/Puppeteer, with full versioning and audit trail logging.

1.4 CCL Summary Specifications Table

Item	Specification & Code Implementation
CCL Purpose	Legally binding engagement letter establishing scope of work, fees, and client terms
Primary User Roles	Caseworker, Admin, Candidate Client
Related Entities	Candidate (User), Case, CandidateApplication, Organisation, VisaType
Output Format	HTML (draft preview) and PDF (issued document download)
Template Source	Database CclTemplate model (HTML + Mustache-style {{tags}}) or imported .docx
Approval Required	Yes — Custom fee proposals require Admin review; issued CCL requires Client acceptance
Audit Logging	Yes — Tracked via CaseCclRecord, audit_logs table, and timeline events
Download Option	Yes — Direct PDF download available to Candidate, Caseworker, Admin, and Superadmin

2. Complete CCL Generation Workflow

The CCL lifecycle encompasses template authoring, fee proposal, draft generation, client issuance, client acceptance/signature, fee payment, and submission gate clearance. The table below documents every step from front to back.

Step	User Action	Frontend Page	API / Route	Backend Logic	Result
1	Log in as Caseworker	LoginPage.jsx	POST /api/auth/login	Authenticate & yield JWT + tenant context	User session established
2	Open Assigned Case	Caseworker Clients	GET /api/cases/:caseId	Fetch case details & check caseworker assignment	Case details loaded
3	Set CCL Fee / Proposal	CclFeeProposalModal	POST /api/ccl-fee-proposal	Validate fee & create pending CaseCclRecord	Fee proposal logged
4	Select / Load Template	CaseDetailCcl.jsx	GET /api/ccl/cases/:caseId	Resolve template by visaTypeId or fallback default	Draft HTML returned
5	Edit or Import Draft	CclTemplateEditor	PUT /api/ccl/cases/:caseId/draft	Store custom draftHtml in CaseCclRecord	Draft saved
6	Preview CCL PDF	CaseDetailCcl.jsx	POST /api/ccl/cases/:caseId/preview	Interpolate {{tags}} & render pdfmake stream	Inline PDF preview
7	Issue CCL to Candidate	CaseDetailCcl.jsx	POST /api/ccl/cases/:caseId/issue	Compile PDF, save file, update status to 'issued', dispatch email	Notification & email sent
8	Candidate Portal View	CandidateCCL.jsx	GET /api/workflow/ccl/status	Fetch issued CCL details for logged-in candidate	Candidate views CCL

Step	User Action	Frontend Page	API / Route	Backend Logic	Result
9	Candidate Accepts CCL	CandidateCCL.jsx	POST /api/workflow/ccl/accept	Update CaseCclRecord status to 'signed', log timeline event	CCL Signed & gate cleared

2.1 Visual Workflow Sequence

[Caseworker/Admin] -> Propose Fee / Select Visa Type



[cclGenerator.service] -> Resolve CclTemplate (Visa-Specific or Default)



[cclTags.service] -> Interpolate {{tags}} with Candidate/Case Data



[Caseworker/Admin] -> (Optional) Edit Draft HTML / Import .docx



[pdfGenerator.service] -> Render PDF via html-to-pdfmake & Sharp Logo Engine



[storage/ccl] -> Save Deterministic PDF & Create Document Record



[Notification Service] -> Issue In-App Alert & Dispatched Email to Candidate



[Candidate Client] -> Accept / Sign CCL -> Status 'signed' -> Submission Gate Unlocked

3. CCL Data Sources & Tag Mapping

The CCL engine resolves data dynamically from multiple database models via buildCclContext() in cclTags.service.js. Candidate-supplied text is sanitized and HTML-escaped to prevent injection.

Tag / Placeholder	Source Table/Model	Source Column	Type	Sample Output Value
{{org_name}}	Organisation	name	text	Elite Immigration Services Ltd
{{org_logo}}	Organisation	logo_url	block	[Rendered PNG Data URI]
{{org_address}}	Organisation	address / company_address	text	100 City Road, London, EC1Y 2BP
{{org_email}}	Organisation	email / primaryEmail	text	info@eliteimmigration.co.uk

Tag / Placeholder	Source Table/Model	Source Column	Type	Sample Output Value
{{org_phone}}	Organisation	phone	text	+44 20 7946 0912
{{candidate_name}}	User (Candidate)	first_name + last_name	text	Arthur Pendelton
{{candidate_first_name}}	User (Candidate)	first_name	text	Arthur
{{candidate_email}}	User (Candidate)	email	text	arthur.pendelton@example.com
{{candidate_address}}	CandidateApplication	currentAddress	text	42 Baker Street, London, NW1 6XE
{{candidate_phone}}	User (Candidate)	country_code + mobile	text	+44 7700 900145
{{candidate_dob}}	CandidateApplication	dob	text	15 May 1990
{{passport_number}}	CandidateApplication	passportNumber	text	GB99887766
{{nationality}}	CandidateApplication	nationality	text	British
{{case_ref}}	Case	caseId	text	ORG-OTH26-015
{{visa_type}}	VisaType / CandidateApplication	name / visaType	text	Skilled Worker Visa
{{petition_type}}	Case	jobTitle / petitionType	text	Initial Entry Clearance
{{caseworker_name}}	User (Caseworker)	first_name + last_name	text	Sarah Jenkins
{{date_today}}	System Runtime	Date.now()	text	15 September 2026
{{date_issued}}	CaseCclRecord	issuedAt	text	15 September 2026
{{proposed_amount}}	CaseCclRecord	feeAmount	text	£1,500.00
{{total_amount}}	Case	totalAmount	text	£1,500.00
{{fee_amount}}	CaseCclRecord	feeAmount	text	£1,500.00
{{amount_in_words}}	Calculated Helper	threeDigitsToWords(feeAmount)	text	One thousand five hundred pounds
{{installment_plan}}	CaseCclRecord	installmentPlan (JSONB)	block	[Formatted HTML Table]

4. CCL Template Architecture

CCL templates are stored centrally in PostgreSQL under the `ccl_templates` table and managed via tenant-scoped models. The application supports organisation defaults, visa-type specific templates, and per-case draft overrides.

Component	Actual Code Path / Table	Architectural Role & Description
Database Table	<code>ccl_templates</code>	Stores HTML templates with <code>visa_type_id</code> foreign key & <code>is_active</code> flag
Sequelize Model	<code>Server/src/models/tenant/cclTemplate.model.js</code>	Defines schema for <code>bodyHtml</code> , <code>headerHtml</code> , <code>footerHtml</code> , <code>visaTypeId</code>
Case CCL Model	<code>Server/src/models/tenant/caseCclRecord.model.js</code>	Tracks per-case <code>draftHtml</code> , <code>feeAmount</code> , <code>status</code> , and issued document IDs
Template Service	<code>Server/src/services/cclTemplate.service.js</code>	Template resolution logic (visa-specific active -> org default)
Tag Interpolator	<code>Server/src/services/cclTags.service.js</code>	Tag registry, context builder, and HTML sanitizer
Generator Service	<code>Server/src/services/cclGenerator.service.js</code>	Combines draft/template HTML with logo data URI into pdfmake buffer
PDF Engine	<code>Server/src/services/pdfGenerator.service.js</code>	Converts html-to-pdfmake definitions into PDF streams via Sharp & PDFKit
CCL Controller	<code>Server/src/modules/Shared/Ccl/ccl.controller.js</code>	API endpoints for CRUD, preview, draft import, and issuance
Frontend Editor	<code>EPiC_Frontend/src/pages/caseworker/CaseworkerCclTemplates.jsx</code>	React rich-text editor interface with live tag insertion palette

5. How the CCL Template is Rendered

Rendering follows a precise content precedence pipeline in `generateCclHtmlForCase()`:

1. Per-Case Draft Override (ccl.draftHtml): If a Caseworker or Admin has explicitly edited or imported a bespoke .docx letter for this case, the draft HTML takes precedence.
2. Visa-Specific Active Template: If no draft exists, the system queries CclTemplate for an active template matching the case's visaTypeId.
3. Organisation Default Template: If no visa-specific template exists, the active template with visaTypeId NULL is loaded.
4. Interpolation & Data Sanitization: Mustache-style tags {{tag}} are replaced with escaped HTML text values or trusted block tables.
5. PDF Compilation: Sharp normalises the organisation logo to PNG data URI. html-to-pdfmake parses the HTML structure into pdfmake layout definitions, compiled to PDF buffer via PDFKit.

6. Caseworker CCL Template Management User Guide

Caseworkers have full operational authority to create, edit, preview, and set active templates for their organisation.

Step-by-Step Template Management Instructions:

1. Sign in to the Caseworker Portal with valid credentials.
2. Navigate to the Left Navigation Bar and select 'CCL Templates' (or Settings -> CCL Templates).
3. View existing templates grouped by Visa Type (e.g. Skilled Worker, Student, Partner, Organisation Default).
4. To Edit an Existing Template: Click the 'Edit' button next to the target template.
5. Modify Body HTML: Use the rich text editor to adjust legal wording, standard clauses, and payment instructions.
6. Insert Dynamic Tags: Click tags from the Tag Palette (e.g. {{candidate_name}}, {{case_ref}}, {{fee_amount}}) to insert placeholders.
7. Preview Live Document: Click 'Preview PDF' to render a sample PDF compiled with your firm's logo and sample candidate data.
8. Set Active Status: Toggle 'Is Active' to make this template the default for that visa category (automatically deactivates older revisions for the same slot).
9. Save Template: Click 'Save Template' to commit changes to the tenant database.
10. Verify Per-Case Issuance: Open an active case and verify the updated template populates when generating a new CCL.

7. Role and Permission Matrix (RBAC)

Access to CCL resources is strictly controlled by authStack.middleware.js and role.middleware.js.

Action / Feature	Candidate	Caseworker	Supervisor	Admin	Superadmin
View Template List	Not Allowed	Allowed	Allowed	Allowed	Allowed
Create / Edit Template	Not Allowed	Allowed	Allowed	Allowed	Allowed
Delete Template	Not Allowed	Allowed	Allowed	Allowed	Allowed
Propose Custom Fee	Not Allowed	Allowed	Allowed	Allowed	Allowed
Approve Custom Fee	Not Allowed	Not Allowed	Allowed	Allowed	Allowed
Issue CCL to Client	Not Allowed	Allowed (Assigned)	Allowed	Allowed	Allowed
View / Accept / Sign CCL	Allowed (Own)	Allowed	Allowed	Allowed	Allowed

8. Security and Data Isolation Policies

1. Multi-Tenant Database Isolation: Every query against `ccl_templates` and `case_ccl_records` is bound to `req.tenantDb`.
2. Caseworker Assignment Scope: Caseworkers can only view and issue CCLs for cases assigned to them via `ensureAssignedCaseCaseworker` middleware.
3. Cross-Tenant Tamper Protection: Query parameters like `?organisation_id` or `?candidateId` supplied in request URLs are strictly ignored in favor of JWT authenticated session context.
4. HTML Injection Prevention: Candidate inputs (names, addresses) are passed through `escapeHtml()` before tag interpolation.

9. API Endpoint Reference

Endpoint Route	HTTP Method	Required Role	Description & Purpose
<code>/api/ccl/templates/tags</code>	GET	Admin, Caseworker	Fetch supported tag catalogue and editor groups
<code>/api/ccl/templates</code>	GET	Admin, Caseworker	List all CCL templates for current tenant
<code>/api/ccl/templates</code>	POST	Admin, Caseworker	Create a new CCL template
<code>/api/ccl/templates/:id</code>	PUT	Admin, Caseworker	Update an existing CCL template

Endpoint Route	HTTP Method	Required Role	Description & Purpose
/api/ccl/templates/:id	DELETE	Admin, Caseworker	Delete a CCL template
/api/ccl/templates/preview	POST	Admin, Caseworker	Generate preview PDF stream from unsaved template HTML
/api/ccl/cases/:caseId	GET	Admin, Caseworker	Fetch case CCL status, draft HTML, or interpolated template
/api/ccl/cases/:caseId/draft	PUT	Admin, Caseworker	Save custom per-case draft HTML override
/api/ccl/cases/:caseId/draft/import	POST	Admin, Caseworker	Import uploaded .docx file and convert to editable draft HTML
/api/ccl/cases/:caseId/issue	POST	Admin, Caseworker	Compile PDF, save document, update status to 'issued', send notifications

10. Database Schema and File Storage Model

- Template Storage: ccl_templates table (id, visa_type_id, name, body_html, header_html, footer_html, is_active, created_by, timestamps).
- Case CCL Storage: case_ccl_records table (id, case_id, status, fee_amount, installment_plan, draft_html, issued_document_id, signed_document_id, issued_at, signed_at).
- File Storage Path: Issued PDF files are stored deterministically under storage/ccl/ccl_case_<caseId>.pdf and referenced in documents table.

11. Error Handling and Troubleshooting Matrix

Error Symptom / Message	Probable Cause	User Action	Technical Action
No template available for visa type	No active template matches visaTypeId and no default template (visaTypeId NULL) exists	Contact Admin to create a default template	Run seedCclTemplates(tenantDb) to populate fallback template
Only .docx files can be imported	User uploaded a PDF or legacy .doc file	Upload a valid Microsoft Word .docx file	Mammoth converter accepts .docx buffers only

Error Symptom / Message	Probable Cause	User Action	Technical Action
Logo image missing in rendered PDF	Organisation logo_url points to missing storage path	Re-upload firm logo in Settings	Check resolveLogoDataUri fallback asset at assets/elitepic_logo.png
Client Care Letter has not been issued yet	Candidate attempted to accept CCL before Caseworker issued it	Wait for Caseworker to issue letter	Verify case_ccl_records status is 'issued'
Cannot transition to application_submitted	Submission gate blocked because CCL is not signed or fee unpaid	Ensure CCL is signed and initial fee payment completed	Check assertSubmissionGate() logic in immigration flow

12. QA Testing Checklist & Automated Test Results

Test Suite / ID	Test Description	Expected Behavior	Automated Execution	Status
ccl.test.js - T1	Tag interpolation and registry mapping	All 24 tags interpolate sample data correctly	node --test tests/ccl.test.js	PASS
ccl.test.js - T2	PDF generation from HTML definition	PDFKit produces non-empty buffer	node --test tests/ccl.test.js	PASS
seeders.test.js	Default CCL seeder idempotency	Creates exactly 1 default template without duplicates	node --test tests/seeders.test.js	PASS
immigrationFlow.test.js	Submission gate enforcement	Blocks submission until CCL signed & fee paid	node --test tests/immigrationFlow.test.js	PASS

Test Suite / ID	Test Description	Expected Behavior	Automated Execution	Status
uat_remaining_modules.test.js	CCL Fee Approval Workflow E2E	Creates proposal, approves fee, issues letter, updates status	node --test tests/uat_remaining_modules.test.js	PASS
sendCredentials.test.js	Initial enquiry case & CCL setup	Creates candidate, application, case, and links CCL	node --test tests/sendCredentials.test.js	PASS

13. UI Screenshots & Empirical Evidence

Empirical verification captured during live application UAT testing:

- Screenshot 1: Caseworker Portal — CCL Template Management Screen (CaseworkerCclTemplates.jsx)
- Screenshot 2: Rich Text Template Editor with Tag Insertion Palette (CclTemplateEditor.jsx)
- Screenshot 3: Case Detail CCL Generation Tab (CaseDetailCcl.jsx)
- Screenshot 4: Live PDF Preview Stream (ccl-preview.pdf)
- Screenshot 5: Candidate Portal — Client Care Letter Acceptance & Signing View (CandidateCCL.jsx)
- Screenshot 6: Admin CCL Fee Approvals Inbox (AdminCclFeeApprovals.jsx)

14. Implementation Gaps & Strategic Recommendations

Identified Gap	Risk	Impact	Recommended Solution	Priority
No Template Version History Table	Low	Unable to roll back template to prior revisions	Create ccl_template_versions audit table	Medium
Client Signature Graphic Overlay	Low	Client accepts via button click instead of drawing signature	Integrate HTML5 Canvas signature pad for digital sign	Low
Multi-language Template Switching	Low	Templates are English-only by default	Add language_code column to CclTemplate	Low

Identified Gap	Risk	Impact	Recommended Solution	Priority
Automated Fee Calculation Rules	Low	Fee must be entered manually per proposal	Add fee rule engine based on visa type & dependent count	Medium

15. Final Executive Summary

The Client Care Letter (CCL) generation and template management subsystem in ImCamHub CRM is fully implemented, highly robust, and verified against all functional, security, and RBAC requirements. Caseworkers possess full operational capability to manage visa-specific templates, preview generated letters, import bespoke .docx files, and issue formal engagement letters. Candidate isolation and submission gates are strictly enforced across backend services and database constraints.

Functional Area	Implementation Status	Verification Evidence / Code Path
CCL Generation Engine	Implemented	cclGenerator.service.js & pdfGenerator.service.js
Caseworker Template Management	Implemented	CaseworkerCclTemplates.jsx & ccl.controller.js
Tag Placeholder System (24 Tags)	Implemented	cclTags.service.js & getCclTagRegistry()
Submission Gate Control	Implemented	immigrationFlow.test.js & assertSubmissionGate()
Multi-Tenant Security & Isolation	Implemented	authStack.middleware.js & ccl.routes.js
Candidate Acceptance & Signing	Implemented	workflow.controller.js & CandidateCCL.jsx